Agentic RAG — Detailed Placement Notes

Simple explanations, sample code, workflow understanding, and placement interview preparation based on the uploaded transcript.

1. What is Agentic RAG?

Agentic RAG combines RAG (Retrieval-Augmented Generation) with an autonomous agent. The agent makes intelligent, dynamic decisions about retrieval and generation.

2. Two Core Components

RAG: provides access to external information such as a vector database, API, tool, or database.
Autonomous agent: decides how the information/retrieval process should be used.

3. Traditional RAG

Traditional RAG generally follows a fixed flow: Query → Retrieve → Context → Prompt → LLM → Answer.

4. Agentic RAG

Agentic RAG adds a decision step: Query → Agent decides → Retrieve OR Generate → Answer. The agent can dynamically choose the path.

5. Four Important Agent Decisions

The transcript highlights: when to retrieve, what to retrieve, where to retrieve, and how many times to retrieve. These decisions are central to the agentic behavior.

6. Why LangGraph?

The transcript uses LangGraph to build the agentic workflow. Think of it as a graph where nodes perform work and edges define how execution moves between nodes. Conditional edges choose the next path.

7. State

A workflow needs shared state. The transcript's state contains question, documents, answer, and needs_retrieval. State is the information passed between workflow nodes.

8. Nodes in the Example

There are three main nodes: decide, retrieve, and generate. START goes to decide; decide conditionally goes to retrieve or generate; retrieve goes to generate; generate goes to END.

9. Decision Function

The transcript demonstrates a simple Python decision function. It checks retrieval-related keywords such as what, how, explain, describe, tell me. If present, it sets needs_retrieval=True. The transcript notes that this logic could later be replaced by an LLM.

10. Retrieve Function

If retrieval is selected, the retriever is invoked with the question and the returned documents are stored in state.

11. Generate Function

The generation node reads the question and documents. If documents exist, their page content is combined into context and inserted into a prompt. The LLM is then invoked to generate the answer.

12. Conditional Logic

The conditional function checks state['needs_retrieval']. True returns the retrieve path; false returns the generate path.

13. Sample State Code

from typing import TypedDict, List from langchain_core.documents import Document class AgentState(TypedDict): question: str documents: List[Document] answer: str needs_retrieval: bool

14. Decision Code

```
def decide_retrieval(state):
    question = state["question"].lower()

    keywords = ["what", "how", "explain",
                "describe", "tell me"]

    needs = any(word in question for word in keywords)

    return {"needs_retrieval": needs}
```

15. Retrieval Code

```
def retrieve_documents(state):
    question = state["question"]
    documents = retriever.invoke(question)

    return {"documents": documents}
```

16. Generation Code

```
def generate_answer(state):
    question = state["question"]
    documents = state.get("documents", [])

    context = "\n\n".join(
        doc.page_content for doc in documents
    )

    prompt = f'''
    Based on the following context,
    answer the question.

    Context:
    {context}

    Question:
    {question}
    '''

    response = llm.invoke(prompt)
    return {"answer": response.content}
```

17. Conditional Function

```
def should_retrieve(state):
    if state["needs_retrieval"]:
        return "retrieve"
    return "generate"
```

18. Conceptual Graph

```
START
  ↓
DECIDE
  ■■■ retrieve → RETRIEVE → GENERATE
  ■■■ generate ■■■■■■■■■■■→ GENERATE
                              ↓
                             END
```

19. Conceptual LangGraph Setup

```
workflow = StateGraph(AgentState)

workflow.add_node("decide", decide_retrieval)
workflow.add_node("retrieve", retrieve_documents)
workflow.add_node("generate", generate_answer)
```

```
workflow.add_edge(START, "decide")

workflow.add_conditional_edges(
    "decide",
    should_retrieve,
    {
        "retrieve": "retrieve",
        "generate": "generate"
    }
)

workflow.add_edge("retrieve", "generate")
workflow.add_edge("generate", END)

app = workflow.compile()
```

20. Running the Workflow

initial_state = { "question": "What is LangGraph?", "documents": [], "answer": "", "needs_retrieval": False } result = app.invoke(initial_state) print(result["answer"])

21. Environment Variables

The transcript loads the OpenAI API key from environment variables. Placement point: do not hard-code API keys or commit them to GitHub. Use environment variables/secrets.

22. Vector Store and Retriever

The transcript creates sample Document objects, stores their content in a vector store, and converts the store into a retriever. The exact store name is unclear in the transcript, so the important concept is documents → embeddings/vector store → retriever.

23. Placement Interview Questions

Q1. What is Agentic RAG?
RAG combined with an autonomous agent that dynamically decides how retrieval and generation should happen.

Q2. Agentic RAG vs traditional RAG?
Traditional RAG follows a mostly fixed retrieval-and-generation flow. Agentic RAG adds dynamic decision-making.

Q3. What decisions can an agent make?
When to retrieve, what to retrieve, where to retrieve, and how many times to retrieve.

Q4. Why LangGraph?
It helps represent the workflow using nodes, state, edges and conditional edges.

Q5. What is state?
Shared data passed between workflow nodes, such as question, documents, answer and retrieval flag.

Q6. What is a node?
A unit of work, such as deciding, retrieving or generating.

Q7. What is a conditional edge?
A transition where the next node depends on the current state/decision.

Q8. What does the retriever do?
It finds relevant external documents/context for a query.

Q9. Can the decision be made without an LLM?
Yes. The transcript demonstrates simple Python keyword rules and says this can later be replaced by an LLM.

Q10. What happens after retrieval?
Retrieved documents are stored in state, converted into context, added to a prompt, and sent to the LLM.

Q11. What happens if retrieval is false?
The workflow can go directly from the decision node to generation.

Q12. What is the role of the agent?
It makes dynamic decisions about actions/retrieval rather than simply following one fixed path.

24. Common Interview Confusions

• Agent ≠ retriever: the retriever fetches information; the agent decides what action/path to take.
• State ≠ vector DB: state carries workflow information; the vector store holds searchable data.
• LangGraph ≠ LLM: LangGraph manages the workflow; an LLM may be used inside nodes for reasoning/generation.
• Conditional edge ≠ normal edge: a conditional edge chooses between paths.

25. Final Placement Cheat Sheet

Agentic RAG = RAG + Autonomous Agent

Traditional: Query → Retrieve → Context → Prompt → LLM
Agentic: Query → Decision → Retrieve OR Generate → Generate

Agent decisions: When? What? Where? How many times?
LangGraph: nodes + state + edges + conditional edges.
Example nodes: decide → retrieve → generate.
State: question, documents, answer, needs_retrieval.

26. 30-Second Interview Answer

> “Agentic RAG combines Retrieval-Augmented Generation with an autonomous agent. Traditional RAG generally follows a fixed retrieve-and-generate pipeline, while Agentic RAG allows an agent to dynamically decide whether retrieval is needed, what or where to retrieve from, and potentially how many retrieval steps are required. In the demonstrated implementation, LangGraph manages the workflow using shared state, nodes and conditional edges such as decide, retrieve and generate.”

27. One-Minute Code Explanation

> “First, I define the workflow state containing the question, documents, answer and retrieval flag. Then I create three nodes: decide, retrieve and generate. The decide node determines whether retrieval is required. A conditional edge checks the flag. If true, the workflow retrieves documents and then generates an answer; otherwise it goes directly to generation. The generation node combines retrieved documents into context and invokes the LLM. Finally, the graph is compiled and invoked with an initial state.”